Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2
По дисциплине «АПО ЭВМиС»
Тема: «Изучение аппаратно-программной архитектуры процессоров семейства Intel – обработка символьных данных».

Выполнил:
Студент 4-го курса
Группы АС-66
Невар В.А.
Проверила:
Кулик

Брест 2026

Цель работы: изучить программную модель микропроцессора; получить практические навыки работы с подсистемой регистровой и оперативной памяти.

Вариант №14

Ход работы

Задание: дан текст – непустая последовательность не длиннее 100 символов, признак конца ввода – точка (в текст не входит). Проверить условие: текст содержит только строчные латинские буквы и цифры, причём буквы и цифры чередуются. Если условие выполнено, поменять местами каждый символ на нечётной позиции и следующий за ним символ (если он есть), иначе выдать сообщение. Ввод, проверку, обработку и вывод оформить отдельными процедурами на ассемблере. Среда разработки – MS Visual Studio (MASM, платформа x64).

Текст программы:

```
; Laboratory work No. 2. Variant 14
; Condition: the text contains only lowercase Latin letters and digits,
;            and letters and digits alternate.
; Rule:      swap every character at an odd position with the next one
;            (if there is one).
;
; MASM, x64, Windows console (Visual Studio, ml64).

OPTION CASEMAP:NONE

includelib kernel32.lib

GetStdHandle PROTO
ReadFile     PROTO
WriteFile    PROTO
ExitProcess  PROTO

STD_INPUT_HANDLE  EQU -10
STD_OUTPUT_HANDLE EQU -11

MAX_TEXT_LEN      EQU 100       ; maximum length of the text
INPUT_BUF_SIZE    EQU 256       ; size of the input buffer
END_MARK          EQU '.'       ; end-of-input mark

.const

msgTitle     db "Laboratory work No. 2. Variant 14", 13, 10, 13, 10, 0
msgCondition db "Condition: the text consists only of lowercase Latin letters
and digits,", 13, 10
             db "           and letters and digits alternate.", 13, 10, 0
msgRule      db "Rule:      if the condition is true, every character at an
odd position", 13, 10
             db "           is swapped with the next character (if there is
one).", 13, 10, 13, 10, 0
msgPrompt    db "Enter a text (1 to 100 characters), end it with a dot '.'
and press Enter:", 13, 10, 0
msgEntered   db 13, 10, "Entered text:     [", 0
msgResult    db "Transformed text: [", 0
```

```
msgClose     db "]", 13, 10, 0
msgCondTrue  db "The condition is TRUE: the text is transformed.", 13, 10, 0
msgCondFalse db "The condition is FALSE: the text is not transformed.", 13,
10, 0
msgNoDot     db 13, 10, "Error: this is not a text - the terminating dot '.'
was not found.", 13, 10, 0
msgEmpty     db 13, 10, "Error: this is not a text - it is empty.", 13, 10, 0
msgTooLong   db 13, 10, "Error: this is not a text - it is longer than 100
characters.", 13, 10, 0

.code

; ------------------------------------------------------------------------
; PrintStr - prints a zero-terminated string to the console.
; In:  rcx = address of the string
; ------------------------------------------------------------------------
PrintStr PROC
    push rbx
    push rsi
    sub  rsp, 38h               ; 20h shadow space + 5th argument + local

    mov  rsi, rcx
    xor  ebx, ebx               ; ebx = string length
ps_len:
    cmp  byte ptr [rsi + rbx], 0
    je   ps_write
    inc  ebx
    jmp  ps_len

ps_write:
    mov  ecx, STD_OUTPUT_HANDLE
    call GetStdHandle
    mov  rcx, rax               ; hFile
    mov  rdx, rsi               ; lpBuffer
    mov  r8d, ebx               ; nNumberOfBytesToWrite
    lea  r9, [rsp + 28h]        ; lpNumberOfBytesWritten
    mov  qword ptr [rsp + 20h], 0   ; lpOverlapped
    call WriteFile

    add  rsp, 38h
    pop  rsi
    pop  rbx
    ret
PrintStr ENDP

; ------------------------------------------------------------------------
; InputText - reads a line from the console and cuts the text at the first
dot.
; In:  rcx = address of the buffer, edx = size of the buffer
; Out: eax = length of the text (the dot is replaced with a zero byte),
;            or -1 if the dot was not found
; ------------------------------------------------------------------------
InputText PROC
    push rbx
    push rsi
    sub  rsp, 38h

    mov  rsi, rcx               ; rsi = buffer
    mov  ebx, edx               ; ebx = buffer size

    mov  ecx, STD_INPUT_HANDLE
```

```
    call GetStdHandle
    mov  rcx, rax               ; hFile
    mov  rdx, rsi               ; lpBuffer
    mov  r8d, ebx               ; nNumberOfBytesToRead
    lea  r9, [rsp + 28h]        ; lpNumberOfBytesRead
    mov  qword ptr [rsp + 20h], 0   ; lpOverlapped
    call ReadFile
    test eax, eax
    jz   it_not_found

    mov  ecx, dword ptr [rsp + 28h] ; ecx = number of bytes read
    xor  eax, eax               ; eax = index in the buffer
it_scan:
    cmp  eax, ecx
    jae  it_not_found
    cmp  byte ptr [rsi + rax], END_MARK
    je   it_found
    inc  eax
    jmp  it_scan

it_found:
    mov  byte ptr [rsi + rax], 0
    jmp  it_exit

it_not_found:
    mov  eax, -1

it_exit:
    add  rsp, 38h
    pop  rsi
    pop  rbx
    ret
InputText ENDP

; ------------------------------------------------------------------------
; CharClass - classifies a character.
; In:  cl = character
; Out: eax = 0 - other, 1 - lowercase Latin letter, 2 - digit
; ------------------------------------------------------------------------
CharClass PROC
    xor  eax, eax
    cmp  cl, '0'
    jb   cc_exit
    cmp  cl, '9'
    jbe  cc_digit
    cmp  cl, 'a'
    jb   cc_exit
    cmp  cl, 'z'
    ja   cc_exit
    mov  eax, 1
    ret
cc_digit:
    mov  eax, 2
cc_exit:
    ret
CharClass ENDP

; ------------------------------------------------------------------------
; CheckCondition - checks that the text consists only of lowercase Latin
; letters and digits, and that letters and digits alternate.
; In:  rcx = address of the zero-terminated text
```

```
; Out: eax = 1 if the condition is true, 0 otherwise
; ------------------------------------------------------------------------
CheckCondition PROC
    push rbx
    push rsi
    sub  rsp, 28h

    mov  rsi, rcx               ; rsi = pointer to the current character
    xor  ebx, ebx               ; ebx = class of the previous character
cn_loop:
    mov  cl, byte ptr [rsi]
    test cl, cl
    jz   cn_true
    call CharClass
    test eax, eax
    jz   cn_false               ; a character of another kind
    cmp  eax, ebx
    je   cn_false               ; two neighbours of the same kind
    mov  ebx, eax
    inc  rsi
    jmp  cn_loop

cn_true:
    mov  eax, 1
    jmp  cn_exit
cn_false:
    xor  eax, eax
cn_exit:
    add  rsp, 28h
    pop  rsi
    pop  rbx
    ret
CheckCondition ENDP

; ------------------------------------------------------------------------
; SwapPairs - swaps every character at an odd position with the next one.
; If the length is odd, the last character stays in place.
; In:  rcx = address of the zero-terminated text
; ------------------------------------------------------------------------
SwapPairs PROC
sw_loop:
    mov  al, byte ptr [rcx]
    test al, al
    jz   sw_exit
    mov  dl, byte ptr [rcx + 1]
    test dl, dl
    jz   sw_exit                ; no next character
    mov  byte ptr [rcx], dl
    mov  byte ptr [rcx + 1], al
    add  rcx, 2
    jmp  sw_loop
sw_exit:
    ret
SwapPairs ENDP

; ------------------------------------------------------------------------
; main - the program entry point.
; Input, check, transformation and output are separate procedures.
; The text buffer is a local array on the stack (no global variables).
; ------------------------------------------------------------------------
main PROC
```

```
    sub  rsp, 128h              ; 20h shadow space + 100h buffer + 8
alignment

    lea  rcx, msgTitle
    call PrintStr
    lea  rcx, msgCondition
    call PrintStr
    lea  rcx, msgRule
    call PrintStr
    lea  rcx, msgPrompt
    call PrintStr

    ; 1. input
    lea  rcx, [rsp + 20h]
    mov  edx, INPUT_BUF_SIZE
    call InputText
    cmp  eax, -1
    je   m_no_dot
    test eax, eax
    jz   m_empty
    cmp  eax, MAX_TEXT_LEN
    ja   m_too_long

    lea  rcx, msgEntered
    call PrintStr
    lea  rcx, [rsp + 20h]
    call PrintStr
    lea  rcx, msgClose
    call PrintStr

    ; 2. check of the condition
    lea  rcx, [rsp + 20h]
    call CheckCondition
    test eax, eax
    jz   m_cond_false

    lea  rcx, msgCondTrue
    call PrintStr

    ; 3. transformation
    lea  rcx, [rsp + 20h]
    call SwapPairs

    ; 4. output of the result
    lea  rcx, msgResult
    call PrintStr
    lea  rcx, [rsp + 20h]
    call PrintStr
    lea  rcx, msgClose
    call PrintStr
    jmp  m_exit

m_cond_false:
    lea  rcx, msgCondFalse
    call PrintStr
    jmp  m_exit

m_no_dot:
    lea  rcx, msgNoDot
    call PrintStr
    jmp  m_exit
```

```
m_empty:
    lea  rcx, msgEmpty
    call PrintStr
    jmp  m_exit

m_too_long:
    lea  rcx, msgTooLong
    call PrintStr

m_exit:
    xor  ecx, ecx
    call ExitProcess
main ENDP

END
```

Результат:

```
Laboratory work No. 2. Variant 14

Condition: the text consists only of lowercase Latin letters and digits,
           and letters and digits alternate.
Rule:      if the condition is true, every character at an odd position
           is swapped with the next character (if there is one).

Enter a text (1 to 100 characters), end it with a dot '.' and press Enter:
a1b2c3d4e5.

Entered text:     [a1b2c3d4e5]
The condition is TRUE: the text is transformed.
Transformed text: [1a2b3c4d5e]

C:\Users\vadim\source\repos\lab2\x64\Debug\lab2.exe (процесс 17224) завершил работу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно:
```

Результат работы программы

Вывод: Я изучил программную модель микропроцессора; получил практические навыки работы с подсистемой регистровой и оперативной памяти при обработке символьных данных на ассемблере.